DA3408 A3: Spark vs. Ray — The Data Engineering Duel

Benchmark Report

Sri Harsha DA24B034

October 5, 2026

1 Objective and setup

This report compares Apache Spark (PySpark) and Ray (Ray Data) on an identical data-cleaning pipeline over the NYC Yellow Taxi trip data. Both frameworks run as a cluster with two workers on a single laptop, so every "node" shares the same CPU, memory and disk. All timings and resource numbers come from my own runs; the Spark UI (port 4040), the Ray Dashboard (port 8265) and the `top` logs back them up.

Machine	Ubuntu laptop, 12 logical cores, 14.9 GiB RAM (4 GiB swap), NVMe SSD
Software	Python 3.14.4, Java 17, PySpark 4.2.0, Ray 2.58.0
Spark cluster	standalone master + 2 workers (4 cores, 3 GiB each); 2 executors × 4 cores, 2 GiB
Spark settings	16 shuffle partitions, session time zone UTC, broadcast join for the lookup
Ray cluster	head (0 CPUs) + 2 workers (4 CPUs each, 1 GB object store each); 8 CPUs in total
Ray settings	hash shuffle V2 (200 partitions), 256 dedupe buckets, spill directory on the NVMe disk, open-file limit raised to 65,535

Table 1: Environment and cluster configuration. The Ray head is started with 0 CPUs so that only the two workers run tasks.

2 Data

The benchmark uses two datasets from the NYC Taxi and Limousine Commission (TLC): the **Yellow Taxi Trip Records** and the **Taxi Zone Lookup** dataset.

2.1 Yellow Taxi Trip Records

The trip dataset consists of monthly Parquet files containing NYC Yellow Taxi trip records. I used the following months:

- **2023:** February–December (11 monthly files)
- **2024:** January–December (12 monthly files)
- **2025:** January–December (12 monthly files)

Thus, the benchmark uses **35 monthly Parquet files** in total. The combined raw dataset contains 125,135,782 rows and occupies approximately 2.0 GB on disk.

The December 2022 and January 2023 files were not included in the final benchmark because their schemas and physical column types were inconsistent with the later files. The raw files were not modified. Instead, the final pipeline handles the schema differences during ingestion by selecting a common set of nine columns required by the benchmark.

The 2025 files additionally contain the column `cbd_congestion_fee`. This column is intentionally excluded during the common-schema selection because it is not required by any subsequent pipeline operation.

2.2 Taxi Zone Lookup

The second dataset is the NYC TLC **Taxi Zone Lookup** CSV, `taxi_zone_lookup.csv`. It contains 265 location records with the following columns:

- `LocationID`
- `Borough`
- `Zone`
- `service_zone`

The lookup table is used to enrich both the pickup and dropoff location IDs in the trip records. All pickup and dropoff location IDs in the retained dataset were found in the lookup table.

Dataset	Size / records
Yellow Taxi Trip Records	35 Parquet files, 125,135,782 rows, ≈2.0 GB
Taxi Zone Lookup	1 CSV file, 265 rows

Table 2: Datasets used in the Spark and Ray benchmark.

A one-off PyArrow scan was used to profile the raw trip data before the benchmark. This profiling was not included in the measured execution time. The following data-quality issues were identified:

Issue found in the raw data	Rows
Pickup timestamp outside 2023-02 to 2025-12 (earliest value 2001)	117
Dropoff not after pickup	574,262
`trip_distance` ≤ 0	2,906,858
Trips shorter than 1 minute (among otherwise valid trips)	464,209
Trips longer than 24 hours	538
Implied speed above 100 mph	41,037
Negative `fare_amount` / `total_amount`	3,936,245 / 1,934,744
Null `passenger_count`, `RatecodeID`	16,940,739
Pickup/dropoff location IDs missing from the lookup	0

Table 3: Data-quality profile of the raw Yellow Taxi trip records.

3 The pipeline (identical logic in both frameworks)

1. **Ingestion:** read all Parquet files from one shared directory.
2. **Schema normalization (immediately after ingestion):** keep nine columns (`VendorID`, pickup/dropoff timestamps, `trip_distance`, `PULocationID`, `DOLocationID`, `payment_type`, `fare_amount`, `total_amount`) and cast each to a fixed type. This removes the schema difference between files. `passenger_count` and `RatecodeID` are dropped: they are not needed for any required step and carry the 13.5% null group.
3. **Cleansing:** (a) drop rows with a null in any kept column (removes 0 rows, since the null-heavy columns were already dropped); (b) keep pickups in [2023-02-01, 2026-01-01); (c) keep trips of 1 minute to 24 hours; (d) keep `trip_distance` > 0; (e) keep non-negative fare and total; (f) remove exact duplicate rows over the nine columns; (g) format timestamps by deriving `pickup_date`, `pickup_hour` and `trip_duration_min`.
4. **Transformation:** join pickup and dropoff zone to the lookup table (borough and zone), compute `avg_speed_mph` with the same custom Python function in both frameworks, and drop speeds above 100 mph. A second output aggregates trips and average speed by pickup borough and hour.
5. **Export:** write the row-level result and the hourly aggregate as Parquet.

The sanity filters are row-wise, so applying them before the duplicate removal gives the same rows as applying them after, but shuffles far fewer rows. Of the 125,135,782 input rows, 117,707,950 (94.06%) were exported in every run of both frameworks.

The implementations differ only in *how* two steps run. Spark uses `dropDuplicates` and a broadcast join. Ray hashes rows into 256 buckets by pickup second (identical rows share a bucket), removes duplicates inside each bucket after a shuffle, and joins the 265-row lookup inside each batch with PyArrow, so in both frameworks the join is broadcast-style, not a shuffle join.

4 Spark results

4.1 Cluster evidence

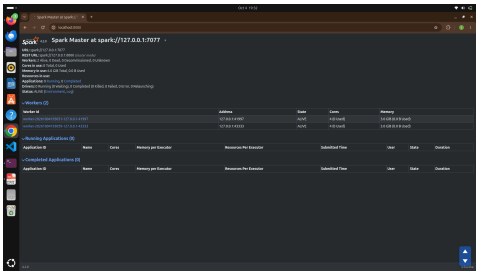


Figure 1: Spark Master UI: two ALIVE workers (4 cores, 3 GiB each) and the completed benchmark applications.

4.2 Execution time and resource usage

Run	Total (s)	Pipeline + export (s)	Hourly aggregate (s)	Peak CPU (%)	Peak memory (GiB)
1	261.1	256.5	4.6	99.4	10.69
2	266.3	261.5	4.8	99.5	10.99
3	253.6	248.8	4.7	99.6	10.88
Mean	**260.3** (sd 6.4)	255.6	4.7	99.5	10.85

Table 4: Spark, full dataset, three timed runs after one warm-up run. Wall-clock time runs from ingestion to export. CPU is `us+sy` and memory is "used" from `top` (1 s samples, whole machine). Memory usage before Run 1 was 4.38 GiB and peaked at 10.69 GiB, an increase of about 6.31 GiB. These are whole-machine "used" values, not Spark-process-only memory.

Peak CPU was about 99.5% in all runs and the run-to-run spread in total time was about 5%.

4.3 Python UDF overhead

The UDF cannot be timed separately inside a lazy pipeline, so I exported the result and then timed the same speed calculation over the exported columns twice: once with the plain Python UDF and once with an equivalent built-in Spark expression. The difference includes the cost of moving rows between the JVM and a Python worker and running Python per row.

	Python UDF (s)	Built-in expr. (s)	Overhead
Full data (117.7 M rows)	53.00, 51.36, 52.71	9.73, 9.65, 9.75	41.71 s (5.3×)

Table 5: Spark UDF overhead (three timed runs each after a discarded warm-up; overhead = best UDF run − best built-in run).

5 Ray results

5.1 Cluster evidence

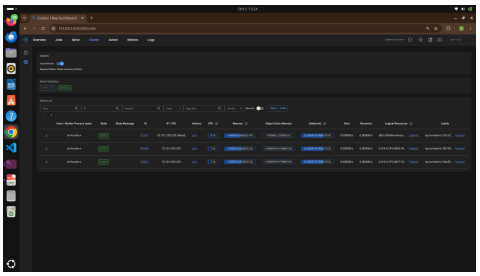


Figure 2: Ray Dashboard (port 8265), cluster view: three nodes (the 0-CPU head and two 4-CPU workers), 8 CPUs.

5.2 Getting a full run to complete

The initial Ray runs encountered two environment-related issues. First, the shuffle exceeded the default open-file limit of 1,024, producing "Too many open files"; raising the limit to 65,535 resolved this. Second, Ray's memory monitor killed workers when memory approached its threshold. Ray was initially spilling to `/tmp`, which is a RAM-backed `tmpfs` on this machine. Moving the spill directory to the NVMe disk and using hash shuffle V2 allowed the full dataset to complete, although some early runs still had worker kills (table below).

The final benchmark therefore used one fixed procedure: restart the Ray cluster, clear the NVMe spill directory, flush swap, record the starting memory, run the pipeline, check all three node logs for memory-pressure kills, and verify output parity. The three counted runs completed without worker kills, with total times of 149.0, 147.8, and 146.7 seconds.

Run	Setup	Total (s)	Worker kills
Warm-up	V2 shuffle, NVMe spill	157.1	yes (2 workers, tasks retried)
Warm-up	+ shuffle-concurrency option*	231.3	none in driver output
Dev. run	same, swap not flushed	230.8	none (all node logs)
Dev. run	same, swap flushed	159.4	yes (all three nodes)
Counted 1–3	final procedure, no option	149.0 / 147.8 / 146.7	none (all node logs)

Table 6: All full-data Ray runs. * An option limiting concurrent shuffle finalizations. The Ray 2.58 source shows it is read only by a different shuffle implementation than the one this plan executes, so I expect it had no effect (not tested directly). It was removed from the final code, and these runs are not counted.

The development runs are not part of the benchmark results. The two ∼231 s runs and the ∼150 s runs differ by about 80 s, and I could not explain the gap from the data I recorded.

Ray remained close to the machine's memory limit during these runs. Whole-machine peak memory was approximately 13.2–13.5 GiB according to 1-second `top` samples. These observations are reported as environmental/resource context rather than as additional benchmark runs.

5.3 Execution time and resource usage

Run	Total (s)	Pipeline (s)	Hourly (s)	Peak CPU (%)	Peak mem. (GiB)	Start mem. (GiB)
1	149.0	118.6	30.3	72.7	13.21	6.62
2	147.8	119.2	28.7	79.8	13.21	6.74
3	146.7	115.2	31.5	78.3	13.22	6.80
Mean	**147.8** (sd 1.2)	117.7	30.2	76.9	13.21	–

Table 7: Ray, full dataset, three counted runs, using the same measurement method as Table 4. Start memory is the "used" value of `free -m` after the cluster was up. Peaks are whole-machine values from `top`.

The Ray runs agree to within 2.3 s (1.6%), tighter than Spark's 5%. The hourly aggregate is slower in Ray (30 s vs. 4.7 s) because it re-reads the exported Parquet and runs another 200-partition shuffle for only 192 groups.

5.4 Python UDF overhead

The same method as for Spark: after the export I timed the speed calculation over the exported columns (117.7 M rows) with the row-wise Python function `speed_mph`, identical to the Spark UDF, and with a vectorized NumPy version (three timed runs after a discarded warm-up).

	Row-wise Python (s)	Vectorized NumPy (s)	Overhead
Full data (117.7 M rows)	28.95, 29.38, 29.63	3.23, 3.42, 3.75	25.72 s (9.0×)

Table 8: Ray UDF overhead (overhead = best row-wise run − best vectorized run).

6 Comparison and UDF discussion

	Spark (mean of 3)	Ray (mean of 3)
Total wall-clock, ingestion to export (s)	260.3 (sd 6.4)	**147.8** (sd 1.2)
Pipeline + export (s)	255.6	117.7
Hourly aggregate (s)	4.7	30.2
Peak CPU, `us+sy` (%)	99.5	76.9
Peak memory, whole machine (GiB)	10.85	13.21
Rows exported	117,707,950	117,707,950
Python function over 117.7 M rows, best of 3 (s)	51.36	28.95
Fast path over 117.7 M rows, best of 3 (s)	9.65 (built-in)	3.23 (NumPy)
Memory-pressure kills in counted runs	0	0

Table 9: Spark vs. Ray on the full dataset. Ray's mean total time is 1.76× shorter.

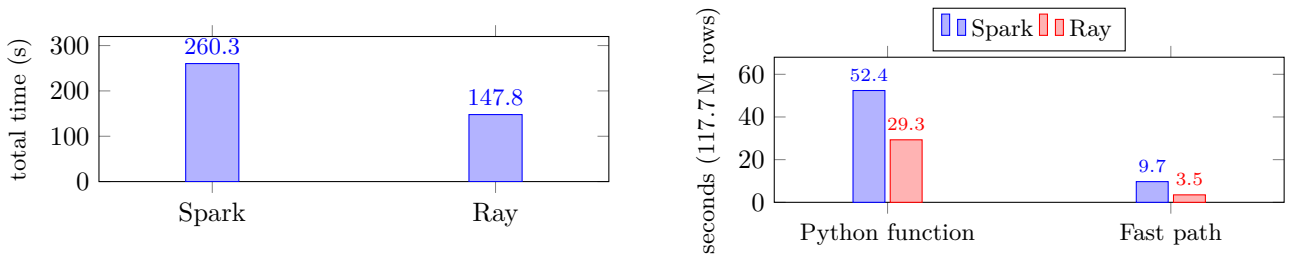


Figure 3: Left: mean total wall-clock time of the full pipeline. Right: mean time of the speed calculation over the exported data (Python function vs. fast path).

6.1 Output parity

For each of the three counted Ray runs I compared its output with the Spark output (`verify_parity.py`): the same 17 columns, the same 117,707,950 rows, equal null counts, equal per-column and whole-row hash checksums, and equal numeric minimum, maximum and sum (tolerance 10^{-9}). The hourly aggregate has the same 192 groups with identical trip counts, and the averages differ by at most 3.1×10^{-13} (relative), i.e. floating-point noise. Spark stores timestamps as nanoseconds and Ray as microseconds; the check compares them as epoch microseconds. The row-level results match exactly, and the hourly averages agree to within floating-point rounding.

6.2 Python-native Ray vs. JVM-based Spark UDF

Applying the *same* Python function over the same 117.7 M values took 28.95 s in Ray and 51.36 s in Spark (1.8× faster). This is consistent with Spark's JVM-to-Python boundary and Python-worker execution overhead, whereas Ray executes the function in Python workers over Arrow batches. I did not isolate that cause experimentally, and the two systems also differ in reader and execution engine, so the gap may include other effects.

Relative to its own fast path, however, the Python penalty is *larger* in Ray (9.0×) than in Spark (5.3×), because Ray's vectorized NumPy path is very fast (3.2 s) and faster than Spark's built-in expression (9.7 s). The two fast paths are not the same kind of thing (a JVM expression versus NumPy in Python), so they should not be compared as like for like.

The row-wise Python results are identical in both frameworks (checksum 1,299,468,753.321), which shows that the function behaves the same. The fast paths each differ slightly from it and from each other (Spark built-in +93.1, Ray NumPy −7.8 on a sum of about 1.3×10^9); I did not establish why, so I do not claim they are numerically equivalent.

6.3 Limits of the comparison

Both clusters run on one laptop, so "nodes" share RAM and disk and Ray's three processes each apply their own memory monitor to the whole machine. Spark's executors were capped at 2 GiB, while Ray's workers had no per-worker heap cap, so the memory configurations are not equivalent.

The Spark runs have only one recorded starting-memory value, whereas for Ray I recorded the baseline of each counted run. Peak CPU and memory are whole-machine values from 1 s samples. The lookup join is small in both frameworks, so this benchmark does not stress a large join.

7 Conclusion: which framework to choose

Winner for this task: Ray, on speed. With identical output, Ray finished the pipeline in 147.8 s against Spark's 260.3 s (1.76× faster), used less peak CPU (76.9% vs. 99.5%), and ran the same Python function 1.8× faster.

The cost was robustness and memory: Ray's whole-machine peak was higher (13.2 vs. 10.9 GiB; the job itself added a similar amount in both, about +6.3 GiB in Spark and +6.4 to +6.6 GiB in Ray, but Ray started from a higher baseline), it operated close to its memory-monitor threshold with swap full, and getting a full run to complete required additional configuration work (raising the file limit, moving spill to disk, and using one fixed run procedure). Spark ran without such recovery events and was more stable in memory.

AI-first project: I would choose Ray. Its Python-native tasks and actors, faster row-wise Python, and Data/Train/Tune libraries suit feature pipelines that feed model training with custom Python code, provided memory is planned (more RAM or a real cluster, not a laptop).

BI-first project: I would choose Spark. SQL and DataFrames, the optimizer, built-in expressions (9.7 s for the calculation that took 51 s as a Python UDF), and predictable memory behavior suit large scheduled aggregations and reporting.

These conclusions come from one dataset on one machine and may not carry over to a multi-node cluster.

8 Performance Tuning Note (use of AI)

I used an LLM assistant for code structure and for the logic translation between PySpark and Ray Data. I built and configured both clusters myself, ran every benchmark myself, and all reported numbers come from those runs (Spark UI, Ray Dashboard and `top` logs).

The assistant suggested the following optimizations, which I reviewed and kept:

- **Early column selection and normalization:** read only nine columns and cast them right after ingestion, which cuts memory and avoids the cross-file schema difference.
- **Filters before duplicate removal:** the row-wise filters run before the shuffle-heavy duplicate removal, giving the same output with less data shuffled.
- **Spark broadcast join:** the 265-row zone table is broadcast to the executors, so the large trip table is not shuffled for the join.
- **UTC session time zone and 16 shuffle partitions:** these provide stable date/hour values and sensible parallelism on the 8 allocated Spark cores.
- **Ray run settings:** the higher open-file limit, NVMe spill directory and hash shuffle V2 were applied after Ray's error messages and disk checks showed that the initial configuration was insufficient for the full dataset.
- **Shuffle-concurrency option, tried and removed:** the assistant suggested limiting concurrent shuffle finalizations. I tested this option during development, but inspection of the Ray 2.58 execution path showed that the executed hash-shuffle plan does not use it. It was therefore removed from the final code and excluded from the counted benchmark results.

The full disclosure is in `AI_DISCLOSURE.md` in the repository.